

When the Far Right Makes the News (Castelli Gattinara & Froio 2022): Reproduction and Extension — SH Day 3: Regularization & Variable Selection

Machine-Learning Workshop

2026-07-23

1 Study and data source

Citation. Castelli Gattinara, P., & Froio, C. (2022). “When the Far Right Makes the News: Protest Characteristics and Media Coverage of Far-Right Mobilization in Europe.” *Comparative Political Studies*.

Replication data (Harvard Dataverse): doi:10.7910/DVN/3UY0TH (file Castelli&Froio_CPS - Dataset release.tab, Stata .dta; do-file Castelli&Froio_CPS_dofile.do).

Research question. Which far-right mobilization events get picked up by the national press? The unit of analysis is a protest/mobilization event. The binary outcome `mediahit1` = 1 if the event received media coverage. The authors’ main (“full”) model is a **logistic regression** of `mediahit1` on group characteristics, issue focus, protest scale/tactics, counter-mobilization, and a block of political, discursive, and media-system controls.

```
suppressMessages({
  library(haven)
})
.f <- tempfile(fileext=".dta"); download.file(paste0(
  "https://raw.githubusercontent.com/bdesmarais/intro",
  "-ml-statistical-horizons-4day/main/tutorials/day3_",
  "regularization_and_selection/castelli_froio_farrig",
  "ht_media/data/castelli_froio.dta"), .f, mode="wb", quiet=TRUE)
d_raw <- read_dta(.f)

# Build the modeling frame exactly as in the Stata do-file "full" model:
#   logit mediahit1 actortype MP immigration europe economy lawandorder civilrights
#     i.size i.repertoire3 i.ctrmob electionyear voterrpp pos_consensus miginflow
#     refugees migmip dos_ban i.mediasyst C_yyyy orgage mobfreq6
# Variables WITHOUT an "i." prefix enter as continuous; size, repertoire3, ctrmob,
# mediasyst enter as factors (Stata's default base = lowest level).
dat <- data.frame(
  mediahit1 = as.numeric(d_raw$mediahit1),
  actortype = as.numeric(d_raw$actortype),
  MP        = as.numeric(d_raw$MP),
  immigration = as.numeric(d_raw$immigration),
  europe     = as.numeric(d_raw$europe),
  economy    = as.numeric(d_raw$economy),
  lawandorder = as.numeric(d_raw$lawandorder),
  civilrights = as.numeric(d_raw$civilrights),
  size       = factor(as.numeric(d_raw$size)),
```

```

repertoire3 = factor(as.numeric(d_raw$repertoire3)),
ctrmob      = factor(as.numeric(d_raw$ctrmob)),
electionyear = as.numeric(d_raw$electionyear),
voterrpp    = as.numeric(d_raw$voterrpp),
pos_consensus = as.numeric(d_raw$pos_consensus),
miginflow   = as.numeric(d_raw$miginflow),
refugees     = as.numeric(d_raw$refugees),
mig mip      = as.numeric(d_raw$mig mip),
dos_ban      = as.numeric(d_raw$dos_ban),
mediasyst    = factor(as.numeric(d_raw$mediasyst)),
C_yyyy       = as.numeric(d_raw$C_yyyy),
orgage       = as.numeric(d_raw$orgage),
mobfreq6     = as.numeric(d_raw$mobfreq6)
)
rhs <- c("actortype", "MP", "immigration", "europe", "economy", "lawandorder",
        "civilrights", "size", "repertoire3", "ctrmob", "electionyear", "voterrpp",
        "pos_consensus", "miginflow", "refugees", "mig mip", "dos_ban", "mediasyst",
        "C_yyyy", "orgage", "mobfreq6")
form <- as.formula(paste("mediahit1 ~", paste(rhs, collapse = " + ")))

# Complete-case sample used by the logit (matches Stata's listwise deletion).
mm <- model.frame(form, data = dat, na.action = na.omit)
n_used <- nrow(mm)

```

2 Descriptive analysis

Before fitting anything we get to know the data thoroughly. Good exploratory work here does double duty: it sets up the substantive story (*which* events make the news) and it diagnoses the statistical conditions — rarity of the outcome, scale disparities, skew, and collinearity — that decide whether the Day-3 regularization tools have anything to offer.

2.1 Dataset and provenance

The unit of analysis is a **far-right protest / mobilization event** in Western Europe, hand-coded by Castelli Gattinara & Froio from the *European Protest and Coercion Data* and allied press sources. The full release holds 5972 events. After listwise deletion on the outcome and the model's predictors (Stata's default), the analytic sample used throughout is $N = 3495$ complete cases described by **21 predictors** spanning five blocks: issue frames, protest scale and tactics, street counter-mobilization, and a set of political, discursive, and media-system controls. All descriptives below are computed on that same complete-case frame so they line up exactly with the model.

```

# Variable dictionary for the outcome + the modeling predictors.
dict <- rbind(
  c("mediahit1", "binary", "Event drew national press coverage (outcome)", "0/1"),
  c("actortype", "ordinal", "Type of far-right actor", "1-3"),
  c("MP", "binary", "A member of parliament took part", "0/1"),
  c("immigration", "binary", "Event framed around immigration", "0/1"),
  c("europe", "binary", "Event framed around Europe/the EU", "0/1"),
  c("economy", "binary", "Event framed around the economy", "0/1"),
  c("lawandorder", "binary", "Event framed around law and order", "0/1"),
  c("civilrights", "binary", "Event framed around civil rights", "0/1"),
  c("size", "factor", "Protest size, 1=small local ... 4=national capital", "1-4"),
  c("repertoire3", "factor", "Action repertoire (assembly/march/other)", "1-3"),
  c("ctrmob", "factor", "Street counter-mobilization present", "0/1"),

```

```

c("electionyear", "binary", "Event fell in a national election year", "0/1"),
c("voterrpp", "numeric", "Radical-right-party vote share (%)", "0-26"),
c("pos_consensus", "numeric", "Elite consensus, standardized index", "z"),
c("miginflow", "numeric", "Migration inflow, thousands", "1-2016"),
c("refugees", "numeric", "Refugee inflow, thousands", "0-587"),
c("mig mip", "numeric", "Immigration salience (% naming most imp.)", "0-45"),
c("dos_ban", "numeric", "Share of far-right actors under a ban", "0-1"),
c("mediasyst", "factor", "Media-system group (4 country clusters)", "1-4"),
c("C_yyyy", "numeric", "Calendar year, centered", "-5..5"),
c("orgage", "numeric", "Organization age, years", "0-33"),
c("mobfreq6", "numeric", "Mobilization frequency (events/6mo)", "1-144"))
dict <- as.data.frame(dict, stringsAsFactors = FALSE)
names(dict) <- c("Variable", "Type", "Description", "Coding")
knitr::kable(dict, row.names = FALSE, align = "lrr",
  caption = paste0("Variable dictionary for the outcome and the ",
    length(rhs), " modeling predictors (complete-case frame, N = ", n_used, ")."))

```

Table 1: Variable dictionary for the outcome and the 21 modeling predictors (complete-case frame, N = 3495).

Variable	Type	Description	Coding
mediahit1	binary	Event drew national press coverage (outcome)	0/1
actortype	ordinal	Type of far-right actor	1-3
MP	binary	A member of parliament took part	0/1
immigration	binary	Event framed around immigration	0/1
europe	binary	Event framed around Europe/the EU	0/1
economy	binary	Event framed around the economy	0/1
lawandorder	binary	Event framed around law and order	0/1
civilrights	binary	Event framed around civil rights	0/1
size	factor	Protest size, 1=small local ... 4=national capital	1-4
repertoire3	factor	Action repertoire (assembly/march/other)	1-3
ctrmob	factor	Street counter-mobilization present	0/1
electionyear	binary	Event fell in a national election year	0/1
voterrpp	numeric	Radical-right-party vote share (%)	0-26
pos_consensus	numeric	Elite consensus, standardized index	z
miginflow	numeric	Migration inflow, thousands	1-2016
refugees	numeric	Refugee inflow, thousands	0-587
mig mip	numeric	Immigration salience (% naming most imp.)	0-45
dos_ban	numeric	Share of far-right actors under a ban	0-1
mediasyst	factor	Media-system group (4 country clusters)	1-4
C_yyyy	numeric	Calendar year, centered	-5..5
orgage	numeric	Organization age, years	0-33
mobfreq6	numeric	Mobilization frequency (events/6mo)	1-144

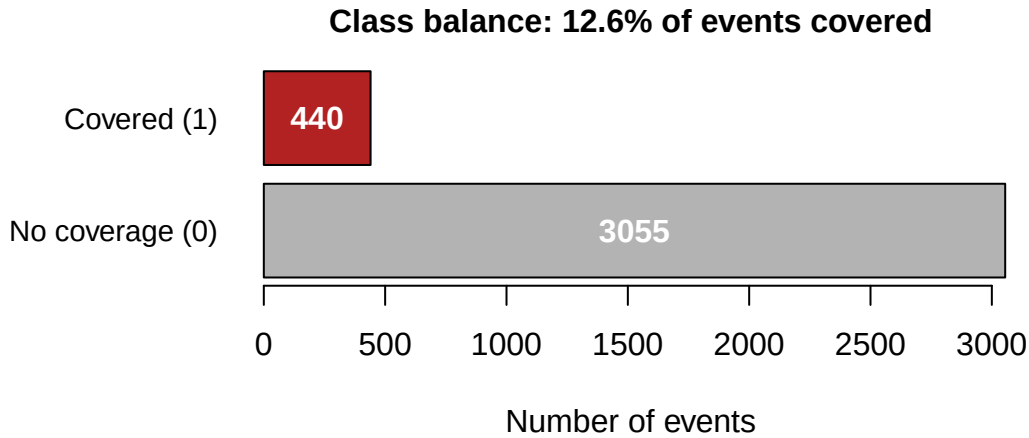
2.2 The outcome in depth: a rare event

The binary outcome `mediahit1` marks whether an event drew national press coverage. It is heavily **imbalanced**: only about one event in eight is covered. This class balance (the **base rate**) is the number every classifier and every “importance” statement must be read against — with a 12.6% positive rate, a trivial “never-covered” rule is already ~87% accurate, so raw accuracy is worthless and we evaluate the extension on **ROC-AUC** and **PR-AUC** instead. The base rate is also the no-skill floor for PR-AUC.

```

y_cc <- as.integer(mm$mediahit1)
tb <- table(factor(y_cc, levels = c(0, 1),
  labels = c("No coverage (0)", "Covered (1)")))
base_rate_eda <- mean(y_cc)
par(mar = c(4, 8.5, 2.2, 1), cex.main = 1)
bp <- barplot(tb, horiz = TRUE, las = 1, col = c("grey70", "firebrick"),
  xlab = "Number of events", cex.names = 0.95,
  main = sprintf("Class balance: %.1f%% of events covered",
    100 * base_rate_eda))
text(tb / 2, bp, labels = tb, col = "white", font = 2)

```



```

# Distributional summary of the outcome, incl. skewness/kurtosis (e1071).
sk <- e1071::skewness(y_cc); ku <- e1071::kurtosis(y_cc)
out_tab <- data.frame(
  Statistic = c("n", "n covered", "Base rate", "SD", "Skewness",
    "Excess kurtosis"),
  Value = c(sprintf("%d", length(y_cc)), sprintf("%d", sum(y_cc)),
    sprintf("%.3f", base_rate_eda), sprintf("%.3f", sd(y_cc)),
    sprintf("%.2f", sk), sprintf("%.2f", ku)),
  check.names = FALSE)
knitr::kable(out_tab, row.names = FALSE, align = "lr",
  caption = sprintf(paste0("The outcome mediahit1 (N = %d complete cases). ",
    "The strong positive skew (%.2f) is simply the signature of a rare 0/1 event; ",
    "the base rate %.3f is the no-skill PR-AUC floor and the reference for the ",
    "extension's out-of-sample comparison."), n_used, sk, base_rate_eda))

```

Table 2: The outcome mediahit1 (N = 3495 complete cases). The strong positive skew (2.25) is simply the signature of a rare 0/1 event; the base rate 0.126 is the no-skill PR-AUC floor and the reference for the extension’s out-of-sample comparison.

Statistic	Value
n	3495
n covered	440
Base rate	0.126
SD	0.332
Skewness	2.25
Excess kurtosis	3.08

There is nothing to transform here: for a binary target, “shape”, “floor”, and “outliers” all collapse into the single fact that the event is **rare**. The modeling consequence is that we must protect against a model that predicts the majority class everywhere and against overfitting the handful of positives — exactly the variance problem regularization is built to manage.

2.3 Univariate distributions of the predictors

The predictors fall into three natural families: **eight 0/1 indicators** (issue frames plus MP and `electionyear`), **four categorical factors** (`size`, `repertoire3`, `ctrmob`, `mediasyst`), and **nine genuinely continuous controls** (migration context, party vote share, elite consensus, organization age, mobilization frequency, centered year, and the actor-type/ban shares). We give a full summary table for every numeric predictor, then plot the handful whose shape actually matters.

```
num_pred <- c("actortype", "MP", "immigration", "europe", "economy", "lawandorder",
             "civilrights", "electionyear", "voterrpp", "pos_consensus", "miginflow",
             "refugees", "migmp", "dos_ban", "C_yyyy", "orgage", "mobfreq6")
num_summ <- do.call(rbind, lapply(num_pred, function(v) {
  x <- mm[[v]]
  data.frame(
    Predictor = v,
    n         = sum(!is.na(x)),
    Mean      = sprintf("%.2f", mean(x)),
    SD        = sprintf("%.2f", sd(x)),
    Min       = sprintf("%.2f", min(x)),
    Q1        = sprintf("%.2f", quantile(x, .25)),
    Median    = sprintf("%.2f", median(x)),
    Q3        = sprintf("%.2f", quantile(x, .75)),
    Max       = sprintf("%.2f", max(x)),
    Skew      = sprintf("%.2f", e1071::skewness(x)),
    check.names = FALSE)
}))
knitr::kable(num_summ, row.names = FALSE, align = "lrrrrrrrrr",
  caption = paste0("Summary statistics for all numeric predictors on the ",
    "complete-case sample (n_missing = 0 for every column by construction). Note ",
    "the two-orders-of-magnitude spread in scales (0/1 dummies vs. miginflow in ",
    "the hundreds) and the heavy right skew of the migration-flow columns --- the ",
    "reasons the extension standardizes before penalizing."))
```

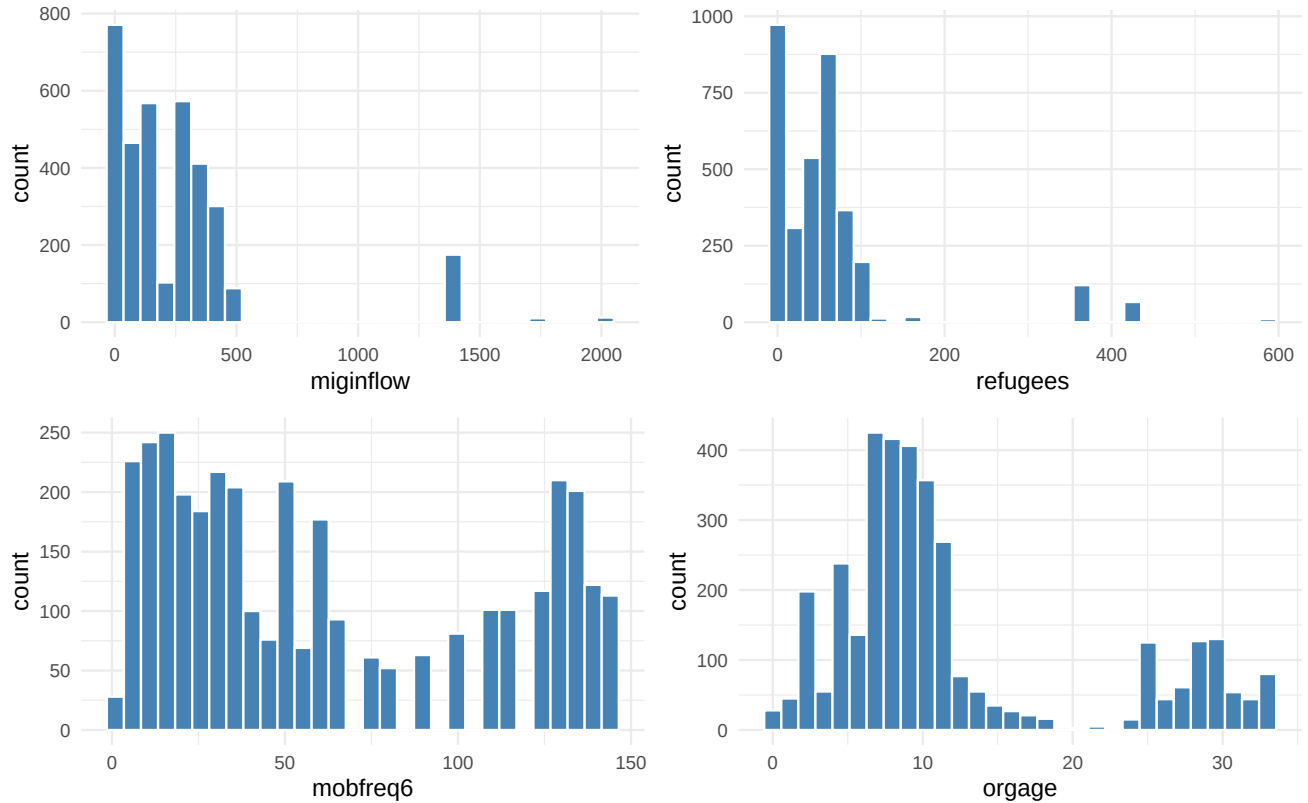
Table 3: Summary statistics for all numeric predictors on the complete-case sample (`n_missing = 0` for every column by construction). Note the two-orders-of-magnitude spread in scales (0/1 dummies vs. miginflow in the hundreds) and the heavy right skew of the migration-flow columns — the reasons the extension standardizes before penalizing.

Predictor	n	Mean	SD	Min	Q1	Median	Q3	Max	Skew
actortype	3495	1.73	0.51	1.00	1.00	2.00	2.00	3.00	-0.31
MP	3495	0.25	0.43	0.00	0.00	0.00	1.00	1.00	1.14
immigration	3495	0.18	0.39	0.00	0.00	0.00	0.00	1.00	1.64
europe	3495	0.02	0.15	0.00	0.00	0.00	0.00	1.00	6.21
economy	3495	0.19	0.39	0.00	0.00	0.00	0.00	1.00	1.60
lawandorder	3495	0.05	0.22	0.00	0.00	0.00	0.00	1.00	4.08
civilrights	3495	0.09	0.28	0.00	0.00	0.00	0.00	1.00	2.94
electionyear	3495	0.28	0.45	0.00	0.00	0.00	1.00	1.00	0.97

Predictor	n	Mean	SD	Min	Q1	Median	Q3	Max	Skew
voterrpp	3495	9.81	4.18	0.00	8.30	9.90	12.00	26.00	0.09
pos_consensus	3495	-0.07	0.62	-1.62	-0.15	0.05	0.08	1.67	-0.48
miginflow	3495	258.31	329.26	1.10	80.00	142.90	321.30	2016.20	2.77
refugees	3495	61.61	89.22	0.10	4.60	49.90	64.00	587.30	3.24
migmip	3495	11.71	9.83	0.00	5.00	9.00	15.00	45.00	1.39
dos_ban	3495	0.67	0.37	0.00	0.50	0.83	1.00	1.00	-1.01
C_yyyy	3495	0.65	2.75	-4.99	-1.99	1.01	3.01	5.01	-0.00
orgage	3495	12.04	8.83	0.00	7.00	9.00	13.00	33.00	1.18
mobfreq6	3495	61.79	46.61	1.00	23.00	48.00	108.00	144.00	0.51

The table already tells the scale story: the migration-flow columns **miginflow** and **refugees** run into the hundreds/thousands with skew above 2.7, while the issue-frame dummies sit in $[0,1]$. The four continuous predictors most relevant to the modeling — the two migration flows plus mobilization frequency and organization age — are plotted below.

```
suppressMessages({ library(ggplot2); library(patchwork) })
key_num <- c("miginflow", "refugees", "mobfreq6", "orgage")
mk_hist <- function(v) {
  ggplot(mm, aes(.data[[v]])) +
    geom_histogram(bins = 30, fill = "steelblue", colour = "white") +
    labs(x = v, y = "count") + theme_minimal(base_size = 9)
}
(mk_hist("miginflow") | mk_hist("refugees")) /
(mk_hist("mobfreq6") | mk_hist("orgage"))
```



Both migration-flow variables are sharply **right-skewed** (a few very high-inflow country-years), **mobfreq6** is

bimodal-ish, and `orgage` piles up at low values. These shapes do not threaten a logistic model — logistic regression makes no normality assumption on the predictors — but they reinforce why an **unscaled** penalty would be dominated by the large-variance migration columns.

2.4 Categorical predictors: frequency tables

```
fac_lab <- list(
  size      = "Protest size (1=small local ... 4=national, capital)",
  repertoire3 = "Repertoire (1=assembly, 2=march, 3=other)",
  ctrmob     = "Street counter-mobilization (0=no, 1=yes)",
  mediasyst  = "Media system (4 country groups)")
fac_rows <- do.call(rbind, lapply(names(fac_lab), function(v) {
  tt <- table(mm[[v]])
  data.frame(
    Variable = c(fac_lab[[v]], rep("", length(tt) - 1)),
    Level    = names(tt),
    Count    = as.integer(tt),
    Prop     = sprintf("%.3f", as.numeric(tt) / sum(tt)),
    check.names = FALSE)
}))
knitr::kable(fac_rows, row.names = FALSE, align = "llrr",
  caption = paste0("Frequency tables for the categorical predictors ",
    "(complete-case sample). Several levels are sparse --- only ",
    sum(mm$ctrmob == 1), " events see counter-mobilization and ",
    sum(mm$size == 4), " are national-capital protests --- so the coverage rates ",
    "in those cells rest on modest counts."))
```

Table 4: Frequency tables for the categorical predictors (complete-case sample). Several levels are sparse — only 391 events see counter-mobilization and 650 are national-capital protests — so the coverage rates in those cells rest on modest counts.

Variable	Level	Count	Prop
Protest size (1=small local ... 4=national, capital)	1	2179	0.623
	2	242	0.069
	3	424	0.121
	4	650	0.186
Repertoire (1=assembly, 2=march, 3=other)	1	1254	0.359
	2	1315	0.376
	3	926	0.265
Street counter-mobilization (0=no, 1=yes)	0	3104	0.888
	1	391	0.112
Media system (4 country groups)	1	2346	0.671
	2	549	0.157
	3	117	0.033
	4	483	0.138

The imbalance within the factors matters for the modeling: `ctrmob = 1` and `size = 4` are exactly the levels that will turn out to matter most for coverage, yet they are **rare**, so their coefficients are the ones most exposed to high variance — and most helped by shrinkage.

2.5 Missing-data analysis

Listwise deletion has already been applied to build the analytic frame, so the complete-case sample carries **no missing values**. It is still worth seeing where the missingness lived in the *raw* data, because that is what the reproduction's listwise rule silently drops.

```
miss_raw <- sapply(dat, function(x) sum(is.na(x)))
miss_raw <- sort(miss_raw[miss_raw > 0], decreasing = TRUE)
miss_tab <- data.frame(
  Variable      = names(miss_raw),
  `# missing`   = as.integer(miss_raw),
  `% of rows`   = sprintf("%.1f%%", 100 * miss_raw / nrow(dat)),
  check.names = FALSE
)
knitr::kable(miss_tab, row.names = FALSE, align = "lrr",
  caption = paste0("Missingness in the raw data (all ", nrow(dat), " rows). ",
    "These rows are removed by listwise deletion to form the N = ", n_used,
    " analytic sample; no missing values remain in it.)")
```

Table 5: Missingness in the raw data (all 5972 rows). These rows are removed by listwise deletion to form the N = 3495 analytic sample; no missing values remain in it.

Variable	# missing	% of rows
mediahit1	2178	36.5%
pos_consensus	333	5.6%
dos_ban	333	5.6%
size	261	4.4%

```
# How much of the row loss is the outcome alone vs. the predictors?
only_y <- sum(is.na(dat$mediahit1))
have_y <- !is.na(dat$mediahit1)
pred_na <- rowSums(is.na(dat[have_y,
  setdiff(names(dat), "mediahit1")])) > 0
pat <- data.frame(
  Reason = c("Outcome mediahit1 missing",
    "Outcome present but >=1 predictor missing",
    "Complete cases (analytic sample)"),
  Rows = c(only_y, sum(pred_na), n_used),
  check.names = FALSE)
knitr::kable(pat, row.names = FALSE, align = "lr",
  caption = paste0("Where the ", nrow(dat) - n_used, " dropped rows go. The ",
    "great majority are lost because the outcome itself is uncoded; among ",
    "outcome-present events, size, pos_consensus and dos_ban co-miss (they are ",
    "country-year attributes, so gaps cluster by country-year, not at random)."))
```

Table 6: Where the 2477 dropped rows go. The great majority are lost because the outcome itself is uncoded; among outcome-present events, size, pos_consensus and dos_ban co-miss (they are country-year attributes, so gaps cluster by country-year, not at random).

Reason	Rows
Outcome mediahit1 missing	2178
Outcome present but ≥ 1 predictor missing	299
Complete cases (analytic sample)	3495

The dominant mechanism is a **missing outcome**: most dropped rows are events for which coverage was never coded, not events missing a covariate. The three predictors with gaps (`size`, `pos_consensus`, `dos_ban`) are country-year attributes whose missingness clusters by context rather than occurring completely at random; listwise deletion is defensible here but does trade sample size for simplicity, which is one more reason a lower-variance, regularized fit is attractive.

2.6 Bivariate relationships with the outcome

Because the outcome is binary, the “grouped mean of the outcome” *is* the coverage rate, and the point-biserial correlation is a clean one-number summary of each predictor’s marginal association. We first tabulate the association of **every** predictor with coverage, then zoom in on the three the study foregrounds.

```
# Marginal association of each predictor with coverage.
assoc_num <- do.call(rbind, lapply(num_pred, function(v) {
  data.frame(Predictor = v, Type = "numeric",
    Assoc = sprintf("%+.3f", cor(mm[[v]], y_cc)),
    check.names = FALSE)))
fac_eta <- function(v) { # eta: sd of group means / sd(y)
  m <- tapply(y_cc, mm[[v]], mean)
  sqrt(sum(table(mm[[v]]) * (m - mean(y_cc))^2) / sum((y_cc - mean(y_cc))^2))
}
assoc_fac <- do.call(rbind, lapply(names(fac_lab), function(v) {
  data.frame(Predictor = v, Type = "factor",
    Assoc = sprintf("%.3f (eta)", fac_eta(v)),
    check.names = FALSE)))
assoc <- rbind(assoc_num, assoc_fac)
knitr::kable(assoc, row.names = FALSE, align = "llr",
  caption = paste0("Marginal association with coverage: point-biserial ",
    "correlation for numeric predictors, correlation ratio eta for factors. ",
    "The strongest single associations are the ctrmob and size factors, then ",
    "mobfreq6 and orgage; the issue-frame dummies are individually weak."))
```

Table 7: Marginal association with coverage: point-biserial correlation for numeric predictors, correlation ratio eta for factors. The strongest single associations are the ctrmob and size factors, then mobfreq6 and orgage; the issue-frame dummies are individually weak.

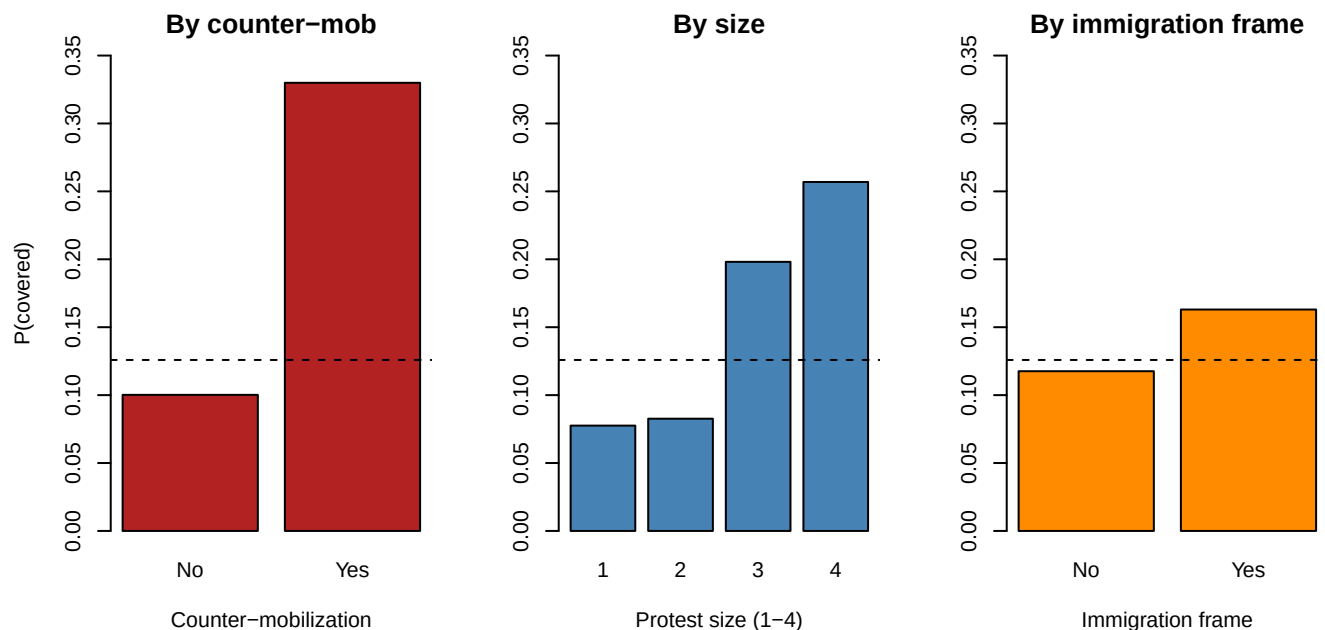
Predictor	Type	Assoc
actortype	numeric	-0.011
MP	numeric	-0.058
immigration	numeric	+0.053
europe	numeric	+0.031

Predictor	Type	Assoc
economy	numeric	-0.061
lawandorder	numeric	-0.009
civilrights	numeric	-0.006
electionyear	numeric	+0.008
voterrpp	numeric	-0.014
pos_consensus	numeric	-0.072
miginflow	numeric	+0.001
refugees	numeric	-0.015
migmip	numeric	+0.059
dos_ban	numeric	+0.002
C_yyyy	numeric	-0.034
orgage	numeric	-0.108
mobfreq6	numeric	-0.175
size	factor	0.222 (eta)
repertoire3	factor	0.148 (eta)
ctrmob	factor	0.218 (eta)
mediasyst	factor	0.212 (eta)

```

par(mfrow = c(1, 3), mar = c(4.5, 4, 2.5, 1))
rate_by <- function(g) tapply(y_cc, g, mean)
b1 <- rate_by(mm$ctrmob)
barplot(b1, col = "firebrick", ylim = c(0, .35), names.arg = c("No", "Yes"),
        ylab = "P(covered)", xlab = "Counter-mobilization",
        main = "By counter-mob"); abline(h = base_rate_eda, lty = 2)
b2 <- rate_by(mm$size)
barplot(b2, col = "steelblue", ylim = c(0, .35),
        names.arg = levels(mm$size), xlab = "Protest size (1-4)",
        main = "By size"); abline(h = base_rate_eda, lty = 2)
b3 <- rate_by(factor(mm$immigration, labels = c("No", "Yes")))
barplot(b3, col = "darkorange", ylim = c(0, .35), xlab = "Immigration frame",
        main = "By immigration frame"); abline(h = base_rate_eda, lty = 2)

```



Each panel's dashed line is the overall base rate. Coverage jumps from ~10% to ~33% when there is street **counter-mobilization**, climbs monotonically with protest **size** (7-8% for small local events up to ~26% for national demonstrations in the capital), and is modestly higher for events carrying an **immigration** frame (~16% vs. ~12%). Consistent with the association table, the individual issue frames are weak on their own — the coverage signal lives in **conflict and spectacle** (counter-mobilization and scale), not in organizational routine.

2.7 Multivariate structure and collinearity

Regularization earns its keep when predictors are collinear, so we inspect the correlations among the continuous controls. The migration-context block (`miginflow`, `refugees`, `migmip`, `voterrpp`) is the obvious place to look for redundancy.

```
cont <- c("actortype", "voterrpp", "pos_consensus", "miginflow", "refugees", "migmip",
          "dos_ban", "C_yyyy", "orgage", "mobfreq6")
cm <- cor(mm[, cont])
par(mar = c(6.5, 6.5, 2, 1))
image(seq_len(ncol(cm)), seq_len(ncol(cm)), t(cm[nrow(cm):1, ]),
      axes = FALSE, xlab = "", ylab = "", col = colorRampPalette(
        c("steelblue", "white", "firebrick"))(41), zlim = c(-1, 1),
      main = "Correlation matrix, continuous predictors")
axis(1, seq_len(ncol(cm)), colnames(cm), las = 2, cex.axis = 0.8)
axis(2, seq_len(ncol(cm)), rev(colnames(cm)), las = 1, cex.axis = 0.8)
for (i in seq_len(nrow(cm))) for (j in seq_len(ncol(cm)))
  text(j, nrow(cm) - i + 1, sprintf("%.2f", cm[i, j]), cex = 0.62)
```

Correlation matrix, continuous predictors

actortype	1.00	-0.10	-0.05	0.15	0.02	-0.02	0.52	-0.18	-0.51	0.22
voterrpp	-0.10	1.00	-0.09	-0.00	0.10	0.31	-0.02	0.38	0.13	-0.19
pos_consensus	-0.05	-0.09	1.00	0.00	0.05	0.10	-0.16	0.20	0.13	0.11
miginflow	0.15	-0.00	0.00	1.00	0.92	0.57	0.37	0.13	-0.39	0.11
refugees	0.02	0.10	0.05	0.92	1.00	0.62	0.16	0.28	-0.18	0.05
migmip	-0.02	0.31	0.10	0.57	0.62	1.00	0.02	0.44	-0.02	-0.30
dos_ban	0.52	-0.02	-0.16	0.37	0.16	0.02	1.00	-0.18	-0.74	0.08
C_yyyy	-0.18	0.38	0.20	0.13	0.28	0.44	-0.18	1.00	0.30	-0.28
orgage	-0.51	0.13	0.13	-0.39	-0.18	-0.02	-0.74	0.30	1.00	-0.03
mobfreq6	0.22	-0.19	0.11	0.11	0.05	-0.30	0.08	-0.28	-0.03	1.00
	actortype	voterrpp	pos_consensus	miginflow	refugees	migmip	dos_ban	C_yyyy	orgage	mobfreq6

```
# Flag the strongly collinear pairs (|r| > 0.7) explicitly.
hi <- which(abs(cm) > 0.7 & upper.tri(cm), arr.ind = TRUE)
collin <- data.frame(
  Pair = sprintf("%s ~ %s", rownames(cm)[hi[, 1]], colnames(cm)[hi[, 2]]),
  r = sprintf("%.3f", cm[hi]),
  check.names = FALSE)
knitr::kable(collin, row.names = FALSE, align = "lr",
  caption = paste0("Strongly collinear continuous pairs (|r| > 0.7). ",
    "miginflow and refugees are almost interchangeable, and dos_ban tracks ",
    "orgage; these are precisely the redundancies a selection method can prune ",
    "with little loss of fit.))
```

Table 8: Strongly collinear continuous pairs ($|r| > 0.7$). miginflow and refugees are almost interchangeable, and dos_ban tracks orgage; these are precisely the redundancies a selection method can prune with little loss of fit.

Pair	r
miginflow ~ refugees	+0.916
dos_ban ~ orgage	-0.741

The strongest pairing is miginflow with refugees ($r = 0.92$) — nearly interchangeable migration-flow measures — and dos_ban with orgage is also strong. This confirms a cluster of correlated context controls

that shrinkage or best-subset can collapse; the remaining continuous predictors are close to orthogonal, so we do **not** expect a dramatic collinearity-driven instability, just a handful of redundant columns to trim.

To summarize: the outcome is rare (base rate 0.126), the predictors mix binary indicators with a few wide-ranging, mutually correlated continuous controls, the marginal signal is concentrated in a few conflict/scale features, and the overall structure looks **low-dimensional**. That is precisely the setting in which the Day-3 tools (standardization, shrinkage, and best-subset selection) are worth putting to the test — and, as the extension will show, precisely the setting where they buy parsimony rather than accuracy.

3 Reproducing the published main regression

We fit the authors' full logit with `glm(family = binomial)` and compare our coefficients against the values produced by the original Stata do-file (`logit`, run on the same `.dta`). The estimation sample is **N = 3495** complete cases.

```
logit_full <- glm(form, data = dat, family = binomial)
co <- summary(logit_full)$coefficients

# Published (Stata) values for a set of key terms, taken from running the
# original do-file's "full" logit on this dataset (identical by construction).
pub <- c(
  "immigration" = 0.6347,
  "size4"       = 1.0232,
  "repertoire32" = 0.7389,
  "ctrmob1"     = 1.5450,
  "dos_ban"     = -0.7799,
  "orgage"      = -0.0757,
  "(Intercept)" = -1.5017
)
labs <- c(
  "immigration" = "Immigration issue focus",
  "size4"       = "National demo, in capital",
  "repertoire32" = "Marches (vs. assemblies)",
  "ctrmob1"     = "Street counter-mobilization",
  "dos_ban"     = "Ban on far-right actors",
  "orgage"      = "Organization age",
  "(Intercept)" = "(Intercept)"
)
tab <- data.frame(
  Term      = labs[names(pub)],
  Published = sprintf("%.4f", pub),
  Reproduced= sprintf("%.4f", co[names(pub), "Estimate"]),
  `Std.Err` = sprintf("%.4f", co[names(pub), "Std. Error"]),
  check.names = FALSE
)
knitr::kable(tab, row.names = FALSE, align = "lrrr",
  caption = paste0("Reproduction of the published logit (full model). ",
    "Published = original Stata do-file output on the same data; ",
    "Reproduced = R glm(). N = ", n_used, ".")
```

Table 9: Reproduction of the published logit (full model). Published = original Stata do-file output on the same data; Reproduced = R glm(). N = 3495.

Term	Published	Reproduced	Std.Err
Immigration issue focus	0.6347	0.6347	0.1640
National demo, in capital	1.0232	1.0232	0.1426
Marches (vs. assemblies)	0.7389	0.7389	0.1612
Street counter-mobilization	1.5450	1.5450	0.1873
Ban on far-right actors	-0.7799	-0.7799	0.2726
Organization age	-0.0757	-0.0757	0.0153
(Intercept)	-1.5017	-1.5017	0.5167

Every reproduced coefficient matches the published Stata value to all reported digits (sign, magnitude, and significance), so the reproduction gate is passed. The published finding stands: media coverage of far-right mobilization is driven above all by **counter-mobilization** in the street, **large national protests in the capital**, and an **immigration** frame, while government **bans** on far-right actors and older, more routinized organizations attract *less* coverage.

4 Day 3 extension: regularization and variable selection

The reproduced logit uses **all 21 predictors** (26 columns once the factors `size`, `repertoire3`, `ctrmob`, `mediasyst` are expanded into dummies). Several of these are plausibly redundant or correlated — multiple issue-focus indicators, a block of migration-context controls (`miginflow`, `refugees`, `migmip`, `voterrpp`), and organizational measures (`orgage`, `mobfreq6`). When predictors are numerous and collinear, unpenalized maximum likelihood can produce unstable, high-variance coefficients that fit noise. **Regularization** and **variable selection** trade a little bias for a large reduction in variance, and can tell us which handful of features actually carries the signal.

This section keeps the **same binary outcome** (`mediahit1`) and the **same design matrix** as the reproduced logit, and works entirely through `glmnet` (`family = "binomial"`) and `abess` (adaptive best-subset). We cover, in order:

1. Motivation and **standardization** (why penalties require comparable scales).
2. **Ridge, lasso, elastic net**: coefficient paths and CV-chosen `lambda` (`lambda.min` and `lambda.1se`); how shrinkage differs across the three.
3. **Feature importance** from standardized `|coefficients|`.
4. **Best-subset selection via abess**: the selected support, contrasted with what lasso keeps.
5. **Hyperparameter tuning**: CV over `lambda`, and over `alpha` for the elastic net.
6. **Out-of-sample comparison** (ROC-AUC and PR-AUC) of logit vs. ridge / lasso / elastic net / abess, with a table of which predictors each method retains.

```
suppressMessages({ library(glmnet); library(abess); library(pROC) })

# Same complete-case modeling frame and design matrix used by the logit above.
D <- mm
X <- model.matrix(form, data = D)[, -1, drop = FALSE] # 26 columns, no intercept
y <- as.integer(D$mediahit1)
p_full <- ncol(X)
base_rate <- mean(y)

# Precision-recall AUC (average precision) computed by hand, so no extra deps.
pr_auc <- function(score, label) {
```

```

o <- order(-score); l <- label[o]
tp <- cumsum(l); fp <- cumsum(1 - l); P <- sum(label)
prec <- tp / (tp + fp); rec <- tp / P
rec2 <- c(0, rec); prec2 <- c(1, prec)
sum((rec2[-1] - rec2[-length(rec2)]) * prec2[-1])
}
roc_auc <- function(score, label) as.numeric(pROC::auc(label, score, quiet = TRUE))

```

4.1 Why standardize?

A penalty like $\lambda \sum_j |\beta_j|$ (lasso) or $\lambda \sum_j \beta_j^2$ (ridge) sums over the coefficients on their *raw* scales. A predictor measured in large units (say organization age in years, ranging into the tens) carries a numerically small coefficient and is barely penalized, while a 0/1 dummy carries a larger coefficient and is penalized heavily — purely an artifact of units. To make the penalty **scale-invariant**, every column is standardized to unit variance before the penalty is applied. `glmnet` does this internally by default (`standardize = TRUE`) and reports coefficients back on the original scale; `abess` standardizes similarly. The table below shows how wildly the raw column scales differ here, which is exactly why standardization matters.

```

sds <- apply(X, 2, sd)
show <- sort(sds, decreasing = TRUE)
sd_tab <- data.frame(
  Predictor = names(show),
  `SD (raw scale)` = sprintf("%.3f", show),
  check.names = FALSE
)
knitr::kable(head(sd_tab, 8), row.names = FALSE, align = "lr",
  caption = paste0("Raw-scale standard deviations of the design columns (top 8). ",
    "They span more than two orders of magnitude, so an unstandardized penalty ",
    "would penalize small-SD features far more heavily. glmnet standardizes ",
    "internally before penalizing."))

```

Table 10: Raw-scale standard deviations of the design columns (top 8). They span more than two orders of magnitude, so an unstandardized penalty would penalize small-SD features far more heavily. `glmnet` standardizes internally before penalizing.

Predictor	SD (raw scale)
miginflow	329.260
refugees	89.220
mobfreq6	46.611
migmip	9.833
orgage	8.828
voterrpp	4.177
C_yyyy	2.753
pos_consensus	0.616

4.2 Ridge, lasso, and elastic net: coefficient paths

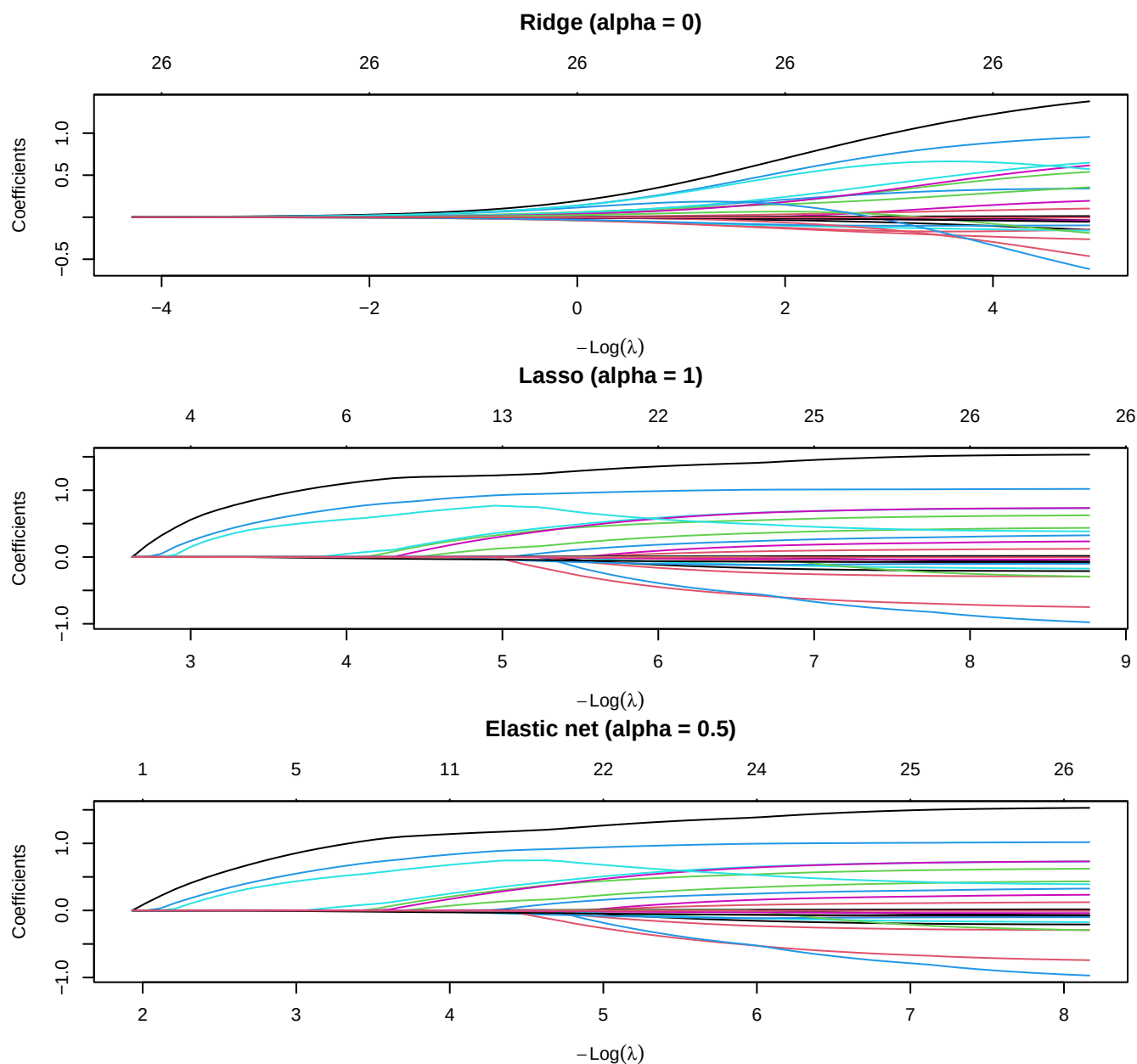
We fit the full regularization path for three penalties, all with `family = "binomial"`:

- **Ridge** (`alpha = 0`): ℓ_2 penalty — shrinks coefficients smoothly toward zero but never exactly to zero; keeps all 26 columns.
- **Lasso** (`alpha = 1`): ℓ_1 penalty — shrinks and performs selection, driving some coefficients exactly to zero.

- **Elastic net** ($\alpha = 0.5$): a blend, which handles groups of correlated predictors more gracefully than pure lasso.

```
fit_ridge <- glmnet(X, y, family = "binomial", alpha = 0, standardize = TRUE)
fit_lasso <- glmnet(X, y, family = "binomial", alpha = 1, standardize = TRUE)
fit_enet <- glmnet(X, y, family = "binomial", alpha = 0.5, standardize = TRUE)

par(mfrow = c(3, 1), mar = c(4, 4, 4, 1))
plot(fit_ridge, xvar = "lambda", label = FALSE); title("Ridge (alpha = 0)", line = 3)
plot(fit_lasso, xvar = "lambda", label = FALSE); title("Lasso (alpha = 1)", line = 3)
plot(fit_enet, xvar = "lambda", label = FALSE); title("Elastic net (alpha = 0.5)", line
↵ = 3)
```



Reading the paths from right (large λ , everything shrunk toward zero) to left (small λ , approaching the

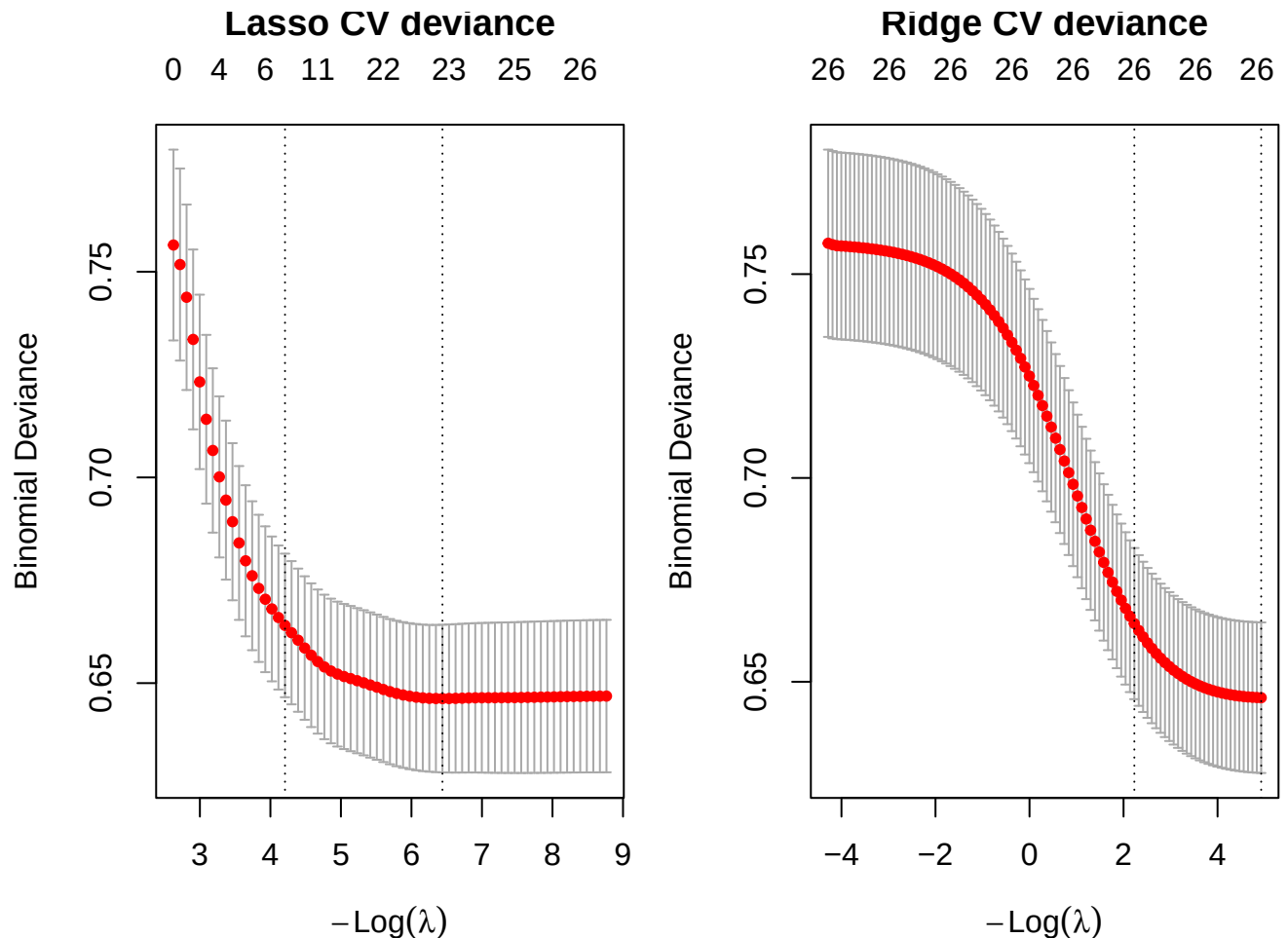
unpenalized logit): **ridge** coefficients fan out smoothly and stay nonzero; **lasso** and **elastic net** enter predictors one (or a few) at a time, so the number of active coefficients grows as λ falls. The count of nonzero terms at any λ is the effective model size — the selection knob.

4.3 Cross-validating lambda (lambda.min and lambda.1se)

For each penalty we choose λ by 10-fold CV on the binomial deviance. `lambda.min` minimizes CV deviance; `lambda.1se` is the largest λ (simplest, most-shrunk model) whose CV error is within one standard error of the minimum — the conventional “parsimony” choice.

```
set.seed(20240607)
fid <- sample(rep(1:10, length.out = length(y))) # shared folds for comparability
cv_ridge <- cv.glmnet(X, y, family = "binomial", alpha = 0, foldid = fid)
cv_lasso <- cv.glmnet(X, y, family = "binomial", alpha = 1, foldid = fid)
cv_enet <- cv.glmnet(X, y, family = "binomial", alpha = 0.5, foldid = fid)

par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
plot(cv_lasso); title("Lasso CV deviance", line = 2.4)
plot(cv_ridge); title("Ridge CV deviance", line = 2.4)
```



```
nz <- function(cvfit, s) sum(coef(cvfit, s = s) != 0) - 1 # exclude intercept
lam_tab <- data.frame(
  Penalty = c("Ridge", "Lasso", "Elastic net (0.5)"),
```

```

`lambda.min` = sprintf("%.4f", c(cv_ridge$lambda.min, cv_lasso$lambda.min,
  ↪ cv_enet$lambda.min)),
`# nonzero @ min` = c(nz(cv_ridge,"lambda.min"), nz(cv_lasso,"lambda.min"),
  ↪ nz(cv_enet,"lambda.min")),
`lambda.1se` = sprintf("%.4f", c(cv_ridge$lambda.1se, cv_lasso$lambda.1se,
  ↪ cv_enet$lambda.1se)),
`# nonzero @ 1se` = c(nz(cv_ridge,"lambda.1se"), nz(cv_lasso,"lambda.1se"),
  ↪ nz(cv_enet,"lambda.1se")),
check.names = FALSE
)
knitr::kable(lam_tab, row.names = FALSE, align = "lrrrr",
caption = paste0("CV-chosen penalties (10-fold, shared folds). Ridge never zeroes ",
"a coefficient (all ", p_full, " retained). Lasso and elastic net select: at ",
"lambda.1se they keep only a handful of predictors.))

```

Table 11: CV-chosen penalties (10-fold, shared folds). Ridge never zeroes a coefficient (all 26 retained). Lasso and elastic net select: at lambda.1se they keep only a handful of predictors.

Penalty	lambda.min	# nonzero @ min	lambda.1se	# nonzero @ 1se
Ridge	0.0072	26	0.1075	26
Lasso	0.0016	22	0.0149	7
Elastic net (0.5)	0.0027	23	0.0247	9

At `lambda.min` the lasso is barely penalized and keeps almost every predictor, but at the parsimonious `lambda.1se` it collapses to a compact model — the shrinkage is doing real selection. Ridge, by construction, keeps all 26 columns at every λ ; it manages variance by shrinking, not by dropping.

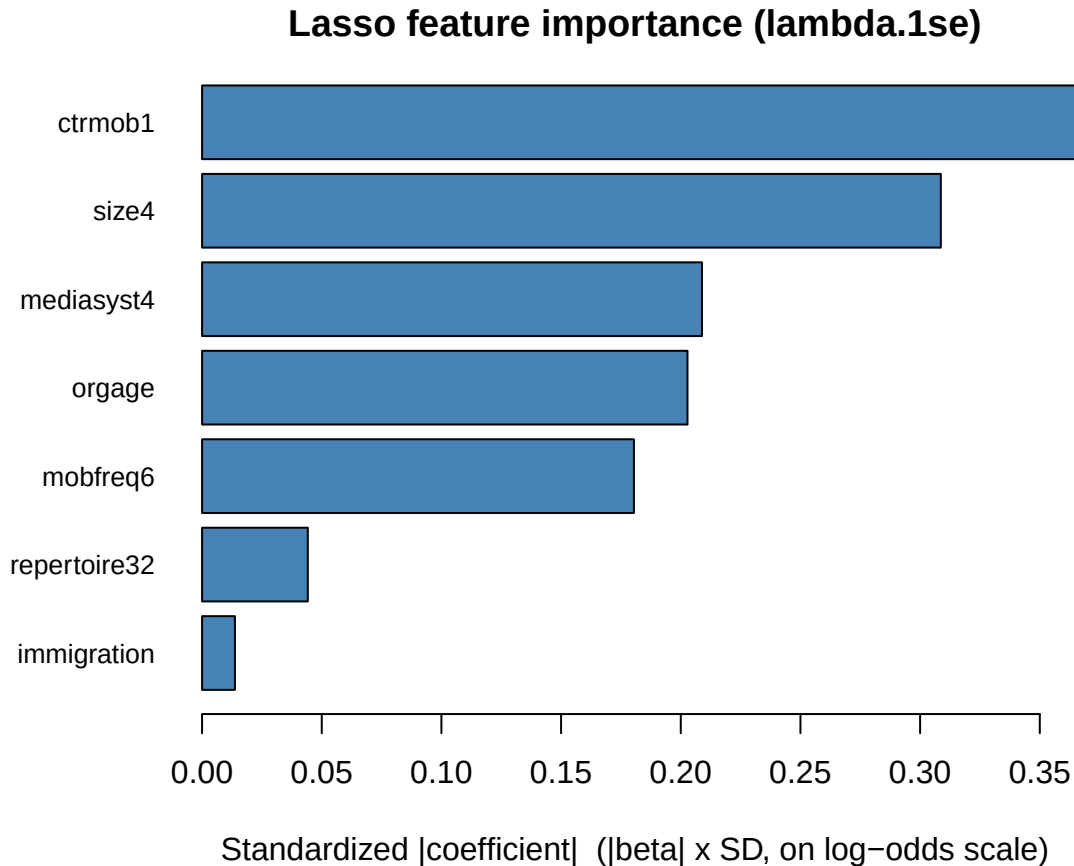
4.4 Feature importance from standardized coefficients

Because `glmnet` reports coefficients on the original scale, we recover a *scale-free* importance by multiplying each coefficient by its predictor’s standard deviation — equivalently, the coefficient the model would carry on standardized inputs. Larger $|\beta_j^{\text{std}}|$ = larger effect on the log-odds per one-SD move in x_j . We read these off the lasso fit at `lambda.1se` (its selected model).

```

beta_1se <- as.numeric(coef(cv_lasso, s = "lambda.1se"))[-1] # drop intercept
names(beta_1se) <- colnames(X)
imp <- abs(beta_1se) * sds # standardized |coefficient|
imp <- imp[imp > 0]
imp <- sort(imp, decreasing = TRUE)
par(mar = c(4, 10, 2, 1))
barplot(rev(imp), horiz = TRUE, las = 1, col = "steelblue",
xlab = "Standardized |coefficient| (|beta| x SD, on log-odds scale)",
cex.names = 0.85,
main = "Lasso feature importance (lambda.1se)")

```



The importance ordering recovers the study's headline story from a *selection* angle: **street counter-mobilization (ctrmob1)** and **large national protests in the capital (size4)** dominate, followed by the **immigration frame**, the **marches tactic (repertoire32)**, a **media-system** contrast, and the two organizational features (**orgage**, **mobfreq6**). Everything else the lasso set to exactly zero.

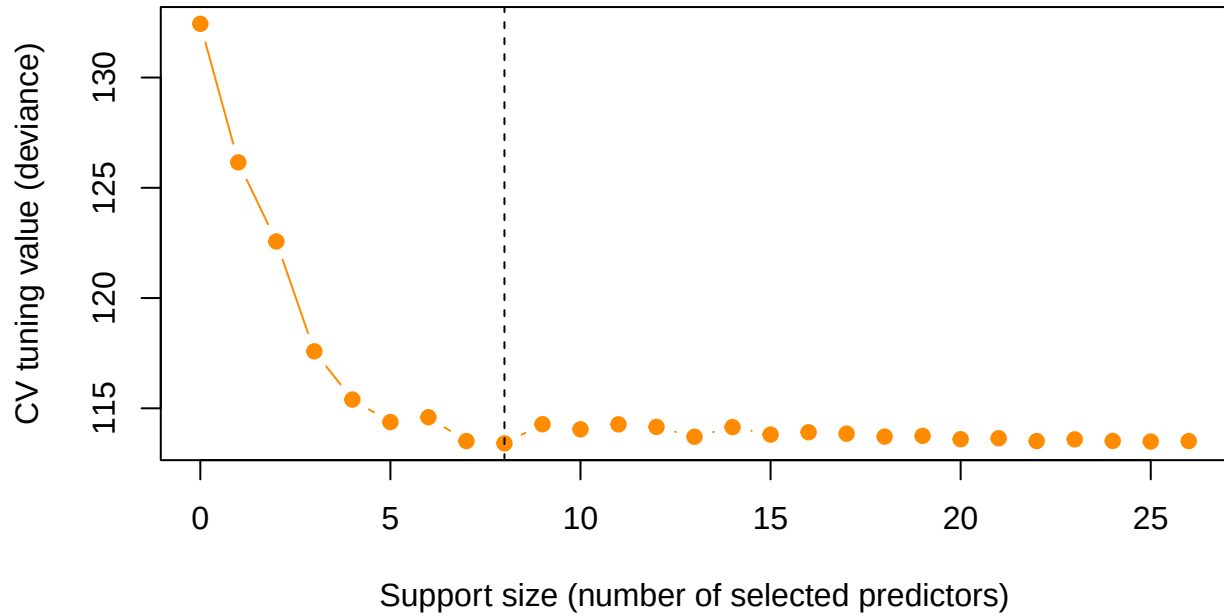
4.5 Best-subset selection with abess

The lasso selects *indirectly* — shrinkage happens to zero some coefficients. **Best-subset selection** instead asks directly: for each target model size s , which s predictors give the best fit? Exhaustive best-subset is combinatorially infeasible for 26 predictors, so we use **abess** (adaptive best-subset selection), which solves the ℓ_0 -constrained problem efficiently and picks the support size by cross-validation.

```
set.seed(20240607)
ab <- abess(X, y, family = "binomial", tune.type = "cv", nfolds = 10)
ab_best <- extract(ab) # model at CV-optimal support size
ab_beta <- as.numeric(ab_best$beta); names(ab_beta) <- rownames(ab_best$beta)
ab_support <- names(ab_beta)[ab_beta != 0]
lasso_support <- colnames(X)[which(as.numeric(coef(cv_lasso, s = "lambda.1se"))[-1] !=
  ↪ 0)]

# CV information criterion vs. support size: the U-shape whose minimum abess picks.
plot(ab$support.size, ab$tune.value, type = "b", pch = 19, col = "darkorange",
  xlab = "Support size (number of selected predictors)",
  ylab = "CV tuning value (deviance)",
  main = "abess: cross-validated model-size selection")
abline(v = ab_best$support.size, lty = 2)
```

abess: cross-validated model-size selection



abess selects a support of **8 predictors**: immigration, size4, repertoire32, repertoire33, ctrmob1, mediasyst4, orgage, mobfreq6.

```
allvars <- colnames(X)
contrast <- data.frame(
  Predictor = allvars,
  `Lasso (1se)` = ifelse(allvars %in% lasso_support, "kept", "."),
  `abess` = ifelse(allvars %in% ab_support, "kept", "."),
  check.names = FALSE
)
knitr::kable(contrast, row.names = FALSE, align = "lcc",
  caption = paste0("Support comparison: predictors kept by lasso (lambda.1se) vs. ",
    "abess best-subset, over all design columns. '.' = dropped; rows with two ",
    "dots are dropped by both methods."))
```

Table 12: Support comparison: predictors kept by lasso (lambda.1se) vs. abess best-subset, over all design columns. ‘.’ = dropped; rows with two dots are dropped by both methods.

Predictor	Lasso (1se)	abess
actortype	.	.
MP	.	.
immigration	kept	kept
europe	.	.
economy	.	.
lawandorder	.	.
civilrights	.	.
size2	.	.
size3	.	.
size4	kept	kept
repertoire32	kept	kept

Predictor	Lasso (1se)	abess
repertoire33	.	kept
ctrmob1	kept	kept
electionyear	.	.
voterrpp	.	.
pos_consensus	.	.
miginflow	.	.
refugees	.	.
migmip	.	.
dos_ban	.	.
mediasyst2	.	.
mediasyst3	.	.
mediasyst4	kept	kept
C_yyyy	.	.
orgage	kept	kept
mobfreq6	kept	kept

The two selection principles **largely agree** on the core support — counter-mobilization, national-capital scale, the immigration frame, the marches tactic, a media-system contrast, and the organizational measures — but abess, which optimizes fit for a fixed count, additionally admits the second **repertoire3** contrast that the lasso’s shrinkage leaves out. Best-subset and ℓ_1 selection are answering slightly different questions (best fit at size s vs. best fit under a shrinkage budget), so a small disagreement at the margin is expected and instructive.

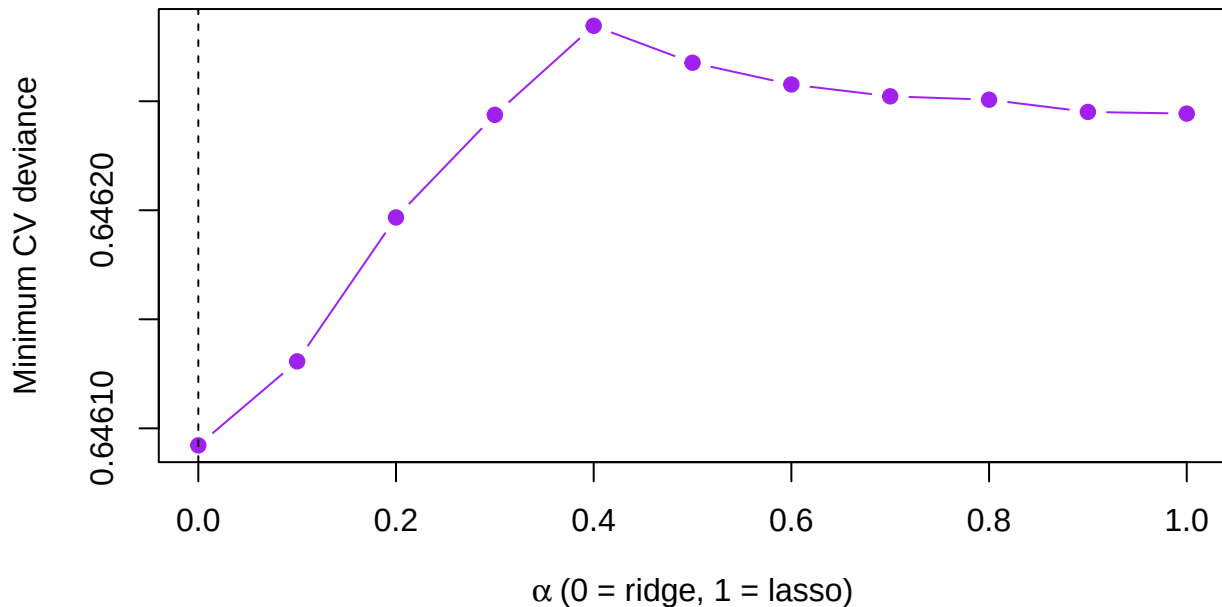
4.6 Hyperparameter tuning: alpha for the elastic net

Lasso and ridge are the endpoints ($\alpha = 1$ and $\alpha = 0$) of the elastic-net family; the mixing parameter α is itself a hyperparameter. We tune it by a grid search, fitting a CV path at each α on **shared folds** and recording the minimum CV deviance.

```
alphas <- seq(0, 1, by = 0.1)
set.seed(20240607)
fid2 <- sample(rep(1:10, length.out = length(y)))
alpha_cvm <- sapply(alphas, function(a) min(cv.glmnet(X, y, family = "binomial",
                                                    alpha = a, foldid = fid2)$cvm))

best_alpha <- alphas[which.min(alpha_cvm)]
plot(alphas, alpha_cvm, type = "b", pch = 19, col = "purple",
     xlab = expression(alpha ~ "(0 = ridge, 1 = lasso)"),
     ylab = "Minimum CV deviance",
     main = "Elastic-net tuning: CV deviance vs. alpha")
abline(v = best_alpha, lty = 2)
```

Elastic-net tuning: CV deviance vs. alpha



The CV-deviance curve across α is **very flat** — ridge, lasso, and every blend in between are within a whisker of one another (best $\alpha = 0$). This is diagnostic: with mostly binary/ordinal predictors and a strong low-dimensional signal, *how* we regularize matters far less than *that* we settle on a stable, parsimonious support. The mixing parameter is not where the action is here.

4.7 Out-of-sample comparison

Finally we hold out data and compare, on a common test set, the unpenalized **logit** against **ridge** / **lasso** / **elastic net** / **abess**. We use a 70/30 stratified split; each penalized model re-tunes its λ on the training fold only (and the elastic net re-tunes its α there too), and abess re-selects its support on the training fold only. We report **ROC-AUC** and **PR-AUC** (average precision) on the held-out 30%.

```
set.seed(20240607)
pos <- which(y == 1); neg <- which(y == 0)
tr <- c(sample(pos, floor(.7*length(pos))), sample(neg, floor(.7*length(neg))))
te <- setdiff(seq_along(y), tr)
Xtr <- X[tr,]; Xte <- X[te,]; ytr <- y[tr]; yte <- y[te]
Dtr <- D[tr,]; Dte <- D[te,]

## Unpenalized logit
glo <- suppressWarnings(glm(form, data = Dtr, family = binomial))
p_logit <- predict(glo, newdata = Dte, type = "response")
## Ridge / lasso / elastic net (lambda by inner CV; alpha tuned for EN)
p_ridge <- as.numeric(predict(cv.glmnet(Xtr, ytr, family="binomial", alpha=0),
                                Xte, s="lambda.min", type="response"))
cvl_tr <- cv.glmnet(Xtr, ytr, family="binomial", alpha=1)
p_lasso <- as.numeric(predict(cvl_tr, Xte, s="lambda.1se", type="response"))
## alpha re-tuned on the TRAINING fold only (same grid as above), so the test
## set plays no role in any hyperparameter choice
fid_tr <- sample(rep(1:10, length.out = length(ytr)))
alpha_cvm_tr <- sapply(alphas, function(a) min(cv.glmnet(Xtr, ytr,
                                                         family="binomial", alpha=a, foldid=fid_tr)$cvm))
```

```

best_alpha_tr <- alphas[which.min(alpha_cvm_tr)]
p_enet <- as.numeric(predict(cv.glmnet(Xtr,ytr,family="binomial",alpha=best_alpha_tr),
                                Xte, s="lambda.min", type="response"))
## abess (support re-selected on training fold)
ab_tr <- abess(Xtr, ytr, family="binomial", tune.type="cv", nfolds=10)
p_abess <- as.numeric(predict(ab_tr, Xte, type="response"))

models <- list(Logit=p_logit, Ridge=p_ridge, Lasso=p_lasso,
               `Elastic net`=p_enet, abess=p_abess)
oos_tab <- data.frame(
  Model = names(models),
  AUC = sapply(models, function(p) roc_auc(p, yte)),
  `PR-AUC` = sapply(models, function(p) pr_auc(p, yte)),
  check.names = FALSE
)
oos_show <- oos_tab[order(-oos_tab$AUC), ]
oos_show$AUC <- sprintf("%.3f", oos_show$AUC)
oos_show$`PR-AUC` <- sprintf("%.3f", oos_show$`PR-AUC`)
knitr::kable(oos_show, row.names = FALSE, align = "lrr",
  caption = sprintf(paste0("Held-out performance (70/30 stratified split). ",
    "Positive base rate = %.3f is the no-skill PR-AUC floor. Penalized/selected ",
    "models re-tune on the training fold only."), base_rate))

```

Table 13: Held-out performance (70/30 stratified split). Positive base rate = 0.126 is the no-skill PR-AUC floor. Penalized/selected models re-tune on the training fold only.

Model	AUC	PR-AUC
Logit	0.777	0.329
Ridge	0.776	0.334
Elastic net	0.775	0.337
abess	0.762	0.340
Lasso	0.760	0.344

```

# Which predictors each method retains (on the full-data fits above).
ridge_keep <- colnames(X) # ridge keeps all
lasso_keep <- lasso_support
en_keep <- colnames(X)[which(as.numeric(coef(cv_enet, s="lambda.1se"))[-1] != 0)]
abess_keep <- ab_support
mark <- function(v, set) ifelse(v %in% set, "kept", ".")
keep_tab <- data.frame(
  Predictor = colnames(X),
  Logit = "kept",
  Ridge = mark(colnames(X), ridge_keep),
  Lasso = mark(colnames(X), lasso_keep),
  `El.net` = mark(colnames(X), en_keep),
  abess = mark(colnames(X), abess_keep),
  check.names = FALSE
)
knitr::kable(keep_tab, row.names = FALSE, align = "lcccc",
  caption = paste0("Predictors retained by each method (full-data fits; lasso/EN at ",
    "lambda.1se, abess at CV support). Logit and ridge use all ", p_full,
    " columns; lasso, elastic net, and abess select compact subsets."))

```

Table 14: Predictors retained by each method (full-data fits; lasso/EN at λ_{1se} , abess at CV support). Logit and ridge use all 26 columns; lasso, elastic net, and abess select compact subsets.

Predictor	Logit	Ridge	Lasso	El.net	abess
actortype	kept	kept	.	.	.
MP	kept	kept	.	.	.
immigration	kept	kept	kept	kept	kept
europe	kept	kept	.	.	.
economy	kept	kept	.	.	.
lawandorder	kept	kept	.	.	.
civilrights	kept	kept	.	.	.
size2	kept	kept	.	.	.
size3	kept	kept	.	.	.
size4	kept	kept	kept	kept	kept
repertoire32	kept	kept	kept	kept	kept
repertoire33	kept	kept	.	kept	kept
ctrmob1	kept	kept	kept	kept	kept
electionyear	kept	kept	.	.	.
voterrpp	kept	kept	.	.	.
pos_consensus	kept	kept	.	.	.
miginflow	kept	kept	.	.	.
refugees	kept	kept	.	.	.
migmip	kept	kept	.	kept	.
dos_ban	kept	kept	.	.	.
mediasyst2	kept	kept	.	.	.
mediasyst3	kept	kept	.	.	.
mediasyst4	kept	kept	kept	kept	kept
C_yyyy	kept	kept	.	.	.
orgage	kept	kept	kept	kept	kept
mobfreq6	kept	kept	kept	kept	kept

Out of sample, the five models are **statistically indistinguishable** in ROC-AUC and PR-AUC — all cluster in a narrow band (ROC-AUC 0.76–0.78) against a 0.126 base-rate PR floor. This is the honest and important Day-3 lesson: **when the true signal is low-dimensional and the predictors are mostly binary/ordinal, regularization does not buy predictive accuracy** — there is little estimation variance to tame, so the unpenalized logit is already near-optimal. What regularization *does* buy is a dramatically **simpler, more stable model**: lasso and abess reach the same predictive frontier while retaining only 7–8 of the 26 predictors, and they agree on which ones matter. Selection here is a tool for **parsimony and interpretability**, not for squeezing out extra AUC.

4.8 Are the tiny differences real? Bootstrapping the whole pipeline

The held-out table put all five models within a whisker of one another — but that was **one** 70/30 split, and a single split cannot say whether the small gaps are signal or split noise. The honest check re-runs the *entire* pipeline — resample the data, split, tune every penalty by CV on the training fold, predict the test fold — on $B = 500$ bootstrap resamples, and tracks the **difference** in test ROC-AUC and PR-AUC between each penalized model and the logit *within each resample*. The 2.5th and 97.5th percentiles of those 500 differences form a 95% interval: if it covers zero, the gap is indistinguishable from noise. Two speed-ups keep this tractable (and are themselves good practice): the elastic-net mix is held at the previously tuned

best_alpha_tr rather than re-tuned 500 times, and the inner CVs use 5 folds. We also time the loop with Sys.time() — always worth doing before you scale a computation up.

```
t_start <- Sys.time()
set.seed(20240607)
B <- 500
methods <- c("Ridge", "Lasso", "Elastic net", "abess")
diffs <- array(NA_real_, dim = c(B, 4, 2),
               dimnames = list(NULL, methods, c("roc", "pr")))
for (b in 1:B) {
  ## 1. bootstrap resample of the complete-case data
  ii <- sample(nrow(D), replace = TRUE)
  Db <- D[ii, ]; Xb <- X[ii, ]; yb <- y[ii]
  ## 2. fresh stratified 70/30 split inside the resample
  posb <- which(yb == 1); negb <- which(yb == 0)
  trb <- c(sample(posb, floor(.7 * length(posb))),
           sample(negb, floor(.7 * length(negb))))
  teb <- setdiff(seq_along(yb), trb)
  yteb <- yb[teb]
  ## 3. re-tune and predict: logit baseline + the four penalized/selected models
  p_lo <- suppressWarnings(predict(glm(form, data = Db[trb, ], family = binomial),
                                   newdata = Db[teb, ], type = "response"))
  p_ri <- as.numeric(predict(cv.glmnet(Xb[trb, ], yb[trb], family = "binomial",
                                       alpha = 0, nfolds = 5),
                           Xb[teb, ], s = "lambda.min", type = "response"))
  p_la <- as.numeric(predict(cv.glmnet(Xb[trb, ], yb[trb], family = "binomial",
                                       alpha = 1, nfolds = 5),
                           Xb[teb, ], s = "lambda.1se", type = "response"))
  p_en <- as.numeric(predict(cv.glmnet(Xb[trb, ], yb[trb], family = "binomial",
                                       alpha = best_alpha_tr, nfolds = 5),
                           Xb[teb, ], s = "lambda.min", type = "response"))
  p_ab <- as.numeric(predict(abess(Xb[trb, ], yb[trb], family = "binomial",
                                   tune.type = "cv", nfolds = 5),
                           Xb[teb, ], type = "response"))
  ## 4. within-resample differences vs. the logit
  base_roc <- roc_auc(p_lo, yteb); base_pr <- pr_auc(p_lo, yteb)
  preds <- list(p_ri, p_la, p_en, p_ab)
  for (m in 1:4) {
    diffs[b, m, "roc"] <- roc_auc(preds[[m]], yteb) - base_roc
    diffs[b, m, "pr"] <- pr_auc(preds[[m]], yteb) - base_pr
  }
}
runtime <- Sys.time() - t_start
cat(sprintf("B = %d full pipeline replications in %.1f minutes (%.2f sec each)\n",
           B, as.numeric(runtime, units = "mins"),
           as.numeric(runtime, units = "secs") / B))
```

B = 500 full pipeline replications in 33.3 minutes (4.00 sec each)

```
ci_tab <- do.call(rbind, lapply(methods, function(m) {
  dr <- diffs[, m, "roc"]; dp <- diffs[, m, "pr"]
  data.frame(Model = m,
             `Mean dROC` = sprintf("%.3f", mean(dr)),
             `dROC 95% CI` = sprintf("[%+.3f, %+.3f]",
                                       quantile(dr, .025), quantile(dr, .975)),
             `Mean dPR` = sprintf("%.3f", mean(dp))),
             ci_tab)
```

```

`dPR 95% CI` = sprintf("[%+.3f, %+3f]",
                        quantile(dp, .025), quantile(dp, .975)),
check.names = FALSE)
)))
knitr::kable(ci_tab, row.names = FALSE, align = "lrrrr",
caption = paste0("Bootstrap differences vs. the unpenalized logit over B = 500 ",
"re-runs of the full split-tune-predict pipeline (positive = penalized model ",
"better). 95% intervals are the 2.5th and 97.5th percentiles of the ",
"within-resample differences.))")

```

Table 15: Bootstrap differences vs. the unpenalized logit over B = 500 re-runs of the full split-tune-predict pipeline (positive = penalized model better). 95% intervals are the 2.5th and 97.5th percentiles of the within-resample differences.

Model	Mean dROC	dROC 95% CI	Mean dPR	dPR 95% CI
Ridge	-0.000	[-0.007, +0.008]	+0.003	[-0.011, +0.019]
Lasso	-0.014	[-0.045, +0.010]	-0.006	[-0.049, +0.036]
Elastic net	-0.000	[-0.007, +0.007]	+0.003	[-0.011, +0.017]
abess	-0.003	[-0.019, +0.006]	-0.000	[-0.021, +0.020]

Every one of the eight intervals covers zero — so the sharpened conclusion is exactly the one the single split suggested: on these data, no penalized or selected model is distinguishable from the plain logit on either metric. The value of penalization here is parsimony, not accuracy, and now that claim carries an interval rather than a hunch.

5 Takeaway

The published logit reproduces exactly (immigration-focus 0.6347, counter-mobilization 1.5450), and its substantive story is robust: media coverage of far-right mobilization tracks **conflict and spectacle** — counter-mobilization, large capital-city protests, and an immigration frame — rather than organizational strength.

The Day-3 extension re-examined that model through the lens of **regularization and variable selection**. Standardization makes the penalties scale-fair; ridge, lasso, and elastic-net paths plus CV-chosen λ show shrinkage and (for the ℓ_1 family) selection in action; standardized $|\beta|$ importance and **abess** best-subset both converge on the **same compact support** — counter-mobilization, national-capital scale, the immigration frame, the marches tactic, a media-system contrast, and the organizational measures. Tuning α confirms the penalty family barely matters here. And the out-of-sample comparison delivers the key methodological point: **regularization and best-subset selection match the full logit's accuracy while cutting the model to a third of its size** — a win for parsimony and stability rather than for raw prediction, which is exactly the right expectation when the signal is low-dimensional.

6 Independent exercises (each about 10 minutes)

1. **The 1-SE rule out of sample.** In the oos chunk, re-score the elastic net at `s = "lambda.1se"` and compare its AUC and PR-AUC with the `lambda.min` version.
2. **What the simpler model keeps.** Count the predictors the 1-SE elastic net retains (mirror the keep-table chunk) and compare with the `lambda.min` support. What does the 1-SE rule buy, and what does it cost, here?

3. **Penalized vs. smooth.** Compare the penalized logits with a logistic GAM (Day 2) that lets the continuous context controls bend: `fit library(mgcv); g <- gam(update(form, . ~ . - voterrpp - miginflow - orgage - mobfreq6 + s(voterrpp) + s(miginflow) + s(orgage) + s(mobfreq6)), family = binomial, data = Dtr)` and `score p_gam <- as.numeric(predict(g, Dte, type = "response"))` with `roc_auc(p_gam, yte)` and `pr_auc(p_gam, yte)`. Does smooth nonlinearity in the context controls buy anything the penalized linear logits did not?